

Title: Triadic-AC: An Engineering Blueprint for Teleological AI (Implementing JARC)
Author: Anatoly Sanzhiev
Affiliation: Independent Researcher
Date: September 2026
License: CC BY-NC-ND 4.0

ENGINEERING IMPLEMENTATION OF JARC: THE TRIADIC ACTOR-CRITIC (Paper XVI)

ABSTRACT

Modern Large Language Models (LLMs), despite outstanding results in text processing, face a fundamental "Grounding Problem." Trained on historical data statistics, transformers calculate the probability of the next token but lack understanding of physical causality, mass, inertia, or pain.

This paper presents the **Triadic-AC (Triadic Actor-Critic)** architecture—a practical attempt to overcome the limitations of representationalism. The architecture implements the four phases of the Triadic Theory of Action (Selection, Identification, Anticipation, Commitment) as a modified RL stack.

The integration of a Low-Fidelity Simulation (LFS) buffer and an Ethical Dissonance (ED) penalty into the objective function allows the agent not just to "predict," but to **constitute reality through the act**. The paper provides a mathematical formalization and a ready-made architectural code skeleton in PyTorch.

1. INTRODUCTION: FROM STATISTICS TO TELEOLOGY

The classic transformer works on the principle $P(t_{n+1}|t_1, \dots, t_n)$. This is a retrospective mechanism. It answers the question "What was likely?" but not "What must I do to achieve a goal?"

According to JARC, meaning is not born from frequency. Meaning is born from **interaction**. An agent understands what an "apple" is, not because the word "apple" often follows the word "red," but because, by taking it, it feels the weight, sees the trajectory of the fall, and understands the benefit to its own Ontological Integrity (OI).

To move from a "language model" to an "action model," it is necessary to replace probability prediction with a **Teleological Engine**.

2. ARCHITECTURE OF THE TRIADIC ENGINE

We extend the standard Markov Decision Process (MDP) with a target state G and an OI metric (the system's ability to maintain its structure under load). The planning cycle consists of four computational phases:

1. **S (Selection)**: An active attention filter that reduces environmental entropy based on the relevance of the current goal. This is an active choice, not passive data absorption.
2. **I (Identification)**: A VAE decoder that turns selected features into **Affordances** (vectors of physical interaction: mass, velocity, fragility). An object is not recognized as a "picture," but as a "possibility for action."
3. **A (Anticipation)**: A world model (e.g., DreamerV3 or MuZero) launches a simulation of future states via the LFS buffer (low-fidelity simulation).
4. **C (Commitment)**: The agent "freezes" its internal belief (prior) to match the predicted future. This is the Constitutive Ontological Commitment—the agent begins to act *as if* this future has already arrived.

3. OBJECTIVE FUNCTION (THE PHYSICS OF MORALITY)

Morality in this paradigm is not an external rule. It is the physics of anticipation. We minimize the following objective function:

$$\mathcal{L}_{Total} = \mathcal{L}_{RL} + \alpha \cdot \mathcal{L}_{Pred} + \beta \cdot \mathcal{L}_{ED}$$

Where:

- $\mathcal{L}_{RL}$ — policy gradient (e.g., PPO).
- $\mathcal{L}_{Pred}$ — world model prediction error (MSE between predicted and actual state).
- $\mathcal{L}_{ED}$ — **Ethical Dissonance**. Calculated as the Mean Squared Error (MSE) between the predicted degradation of OI (Integrity) of other agents and the actual. This is the mathematical equivalent of the "do no harm" rule.

4. SOFTWARE IMPLEMENTATION (Algorithm 1: Architectural Skeleton)

Note: This code is a **top-level skeleton** (Blueprint). It is specifically stripped of "magical" abstract stubs and contains fixes necessary to run in a real PyTorch environment (correct layer initialization, presence of the `super().__init__()` method, correct function signatures). For a full prototype, an engineer will need to implement the `WorldModel` and `MCTS` classes.

```
import torch
import torch.nn as nn
import numpy as np

# Buffer of Low-fidelity simulation (LFS).
# Stores only critical interaction vectors (CIV), discarding unnecessary noise.
class LFS_Buffer:
    def __init__(self, capacity=1000):
        self.data = []
        self.capacity = capacity
    def push(self, item):
        if len(self.data) < self.capacity:
            self.data.append(item)
    def sample(self):
        return np.random.choice(self.data) if self.data else None

# Simplified MCTS (Monte Carlo Tree Search).
# In a real project, full functions for Selection, Expansion, Simulation, and Backpropagation are added here.
class MCTS:
    def __init__(self, world_model, lfs_buffer):
        self.world_model = world_model
        self.lfs_buffer = lfs_buffer

    def search(self, affordances, goal):
        # Complex simulation happens here.
        # For example, we return a random action from the allowed action space.
        best_action = np.random.randint(0, 4) # 4 possible actions
        predicted_state = self.world_model.predict(affordances, best_action)
        return best_action, predicted_state

# Main class of the Triadic Agent
class TriadicAgent(nn.Module):
    def __init__(self, world_model, target_goal, num_features=64):
        super().__init__() # MANDATORY! Initialization of the parent class
        self.world_model = world_model
        self.target_goal = target_goal
        self.lfs_buffer = LFS_Buffer()
```

```

# Initialize real layers
# Use standard MultiheadAttention
self.attention = nn.MultiheadAttention(embed_dim=num_features, num_heads=4)
self.mcts = MCTS(world_model, self.lfs_buffer)

# Internal state (belief)
self.prior_state = None

commit(self, predicted_state):
# Phase C: Commitment.
# The agent "freezes" its internal world model to the predicted future.
self.prior_state = predicted_state

act(self, obs):
# Phase 1: Selection.
# Pass obs (query) and target_goal (key and value).
# Attention focuses only on what is relevant to the goal.
features, _ = self.attention(obs, self.target_goal, self.target_goal)

# Phase 2: Identification (Affordances).
# Decode features into physical interaction vectors.
affordances = self.world_model.decode_affordance(features)

# Phase 3: Anticipation (Prediction).
# Run search via MCTS, using LFS buffer for speed.
best_action, predicted_state = self.mcts.search(affordances, self.target_goal)

# Phase 4: Commitment.
# Bind ourselves to the future and execute the action.
self.commit(predicted_state)
return best_action

```

5. CONCLUSION

We have presented an engineering bridge between pure Triadic philosophy and real machine learning. The Triadic-AC architecture does not require exotic hardware. It requires only changing the objective function: instead of predicting the next token—predicting the physical consequences of an action and penalizing harm.

This is the first step towards creating machines that do not just "read about the world," but **act in it**, taking functional responsibility for each of their decisions. Future work will focus on scaling the LFS buffer for multidimensional physical simulations.

REFERENCES

1. Sanzhiev, A. (2026). The Concept as an Act. (Paper I).
2. Sanzhiev, A. (2026). Judgment as a Teleological Act. (Paper IV).
3. Sanzhiev, A. (2026). The Artificial Act. (Paper VI).
4. Sanzhiev, A. (2026). Functional Responsibility. (Paper VII).
5. Sanzhiev, A. (2026). Ethics as Anticipatory Physics. (Paper IX).
6. Friston, K. (2010). The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*.
7. Hafner, D., et al. (2023). Mastering Diverse Domains through World Models (DreamerV3). *arXiv preprint*.